

Lab. 10

Hardening: Performance & Security

Content

1. Why Queries Get Slow
2. Indexes: what they are and how to create them
3. Reading a query plan: EXPLAIN
4. The cost of indexes, and tuning tips
5. Security threats: injection and over-privilege
6. Parameterised queries as the injection defence
7. Least privilege: a dedicated app user
8. Protecting secrets and errors
9. Running the starter (npm run dev); good habits; next: Lab 11
10. Summary

Recap: Correct, Now Fast and Safe

- Your app works and, since Lab 9, stays correct even across multi-step changes.
- Two things still stand between it and the real world: speed and safety.
- Performance: as tables grow, the wrong queries can crawl. Indexes fix that.
- Security: an exposed app invites attack. We harden access and block injection.

Why Queries Get Slow

The Problem: Full Table Scans

- By default, a SELECT ... WHERE has to check every row; a full table scan.
- With 50 rows that is instant; with 5 million it is painfully slow.
- The cost grows with the number of rows, even when only a few match.
- The fix is to let the database jump straight to the rows it needs, with an index.

Indexes: What They Are and How to Create Them

- An index is like the index at the back of a book: a sorted lookup that finds things fast.
- Instead of scanning every page, the database looks up the value and goes straight there.
- MySQL keeps indexes as balanced trees (like a dictionary search for a word), so a lookup stays fast even on huge tables.
- Your primary key is already indexed, which is why finding a row by id is quick.

Indexes: What They Are and How to Create Them (Contd.)

Creating an Index:

Add an index on the columns you search, join or sort by. The syntax is one line.

```
-- speed up searches by course title
CREATE INDEX idx_courses_title ON courses (title);

-- foreign keys (enrolments.course_id) are indexed automatically
 SHOW INDEX FROM courses;   -- see what exists
```

Index the columns that appear in WHERE, JOIN and ORDER BY, not every column.

Reading a Query Plan: EXPLAIN

EXPLAIN First; Put EXPLAIN in front of any SELECT to see how MySQL will run it, before optimising.

```
EXPLAIN SELECT * FROM courses WHERE title = 'Operating Systems';

-- key columns in the output:
--   type : ALL = full scan (slow), ref/eq_ref = uses an index
--   key  : which index was used (NULL = none)
--   rows : how many rows MySQL expects to examine
```

type = ALL and a big rows count are the warning signs. A good index turns ALL into ref.

Reading a Query Plan: EXPLAIN (Contd.)

Before and After an Index:

Measure, don't guess: EXPLAIN before, add the index, EXPLAIN again, and compare.

```
EXPLAIN SELECT * FROM courses WHERE title = 'Operating Systems';
--  type: ALL    key: NULL    rows: 500

CREATE INDEX idx_courses_title ON courses (title);

EXPLAIN SELECT * FROM courses WHERE title = 'Operating Systems';
--  type: ref    key: idx_courses_title    rows: 1
```

Same query, far fewer rows examined. That is the whole point of an index.

Reading a Query Plan: EXPLAIN (Contd.)

Measuring the Speed-up:

EXPLAIN ANALYZE runs the query and reports the real time. First give the table enough rows to notice.

```
USE css326_lab10;
-- 1. add 100,000 test rows
SET SESSION cte_max_recursion_depth = 100000;
INSERT INTO courses (title, credits)
WITH RECURSIVE n AS (SELECT 1 AS i UNION ALL SELECT i + 1 FROM n WHERE i < 100000)
SELECT CONCAT('Course ', i), 3 FROM n;
-- 2. time it before and after CREATE INDEX
EXPLAIN ANALYZE SELECT * FROM courses WHERE title = 'Operating Systems';
-- before: -> Filter ... (actual time=0.04..30.0 rows=1)   -> Table scan on courses ... (rows=100005)
-- after:  -> Index lookup on courses using idx_courses_title ... (actual time=0.009..0.009 rows=1)
```

The last number in actual time is the total in ms (yours will differ). Run each query twice and read the second result.

Running the Starter Project

From the project folder: open a terminal in Lab10_Exercise_Starter (VS Code: Terminal → New Terminal) and run:

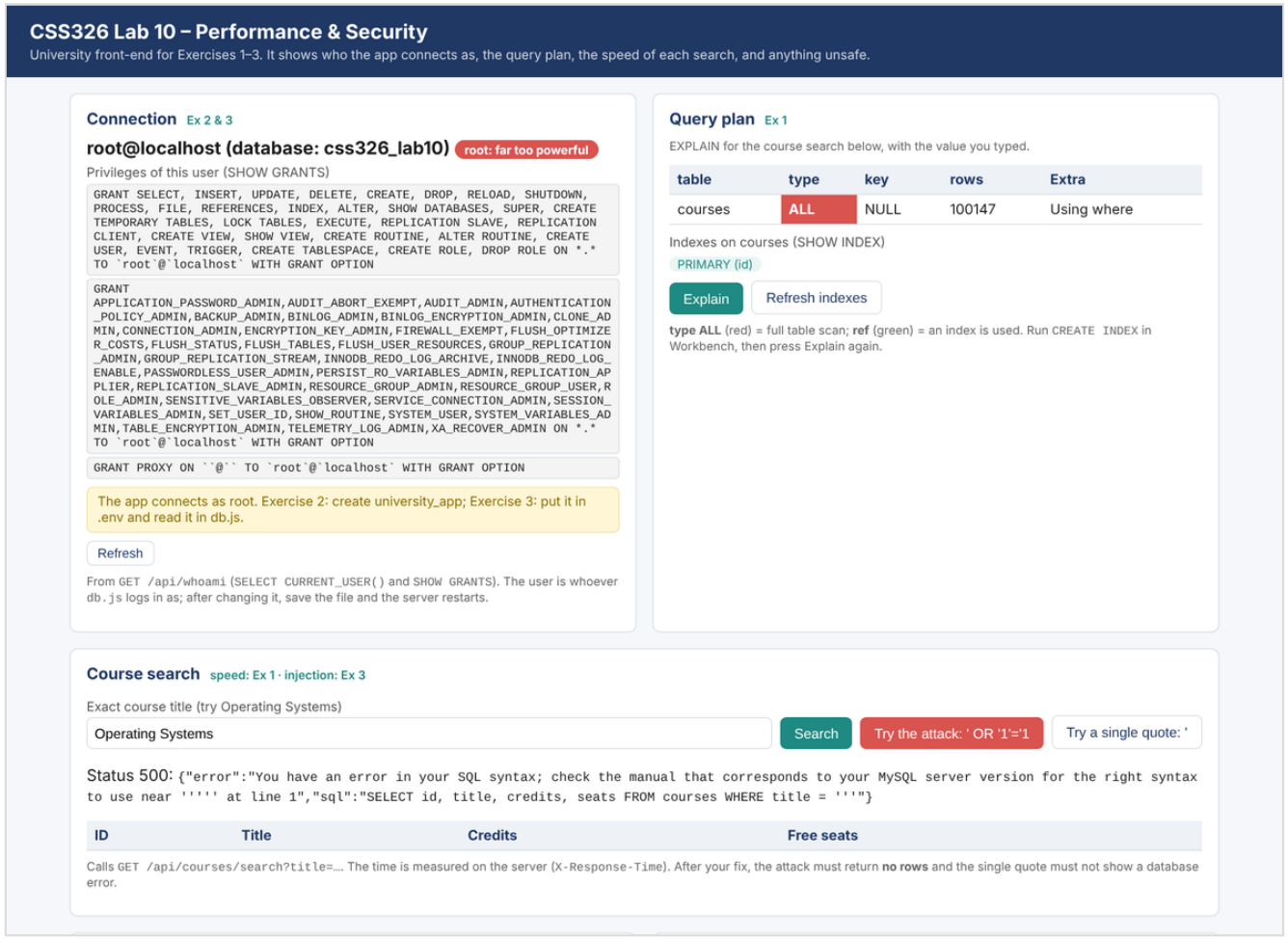
npm install	← once: downloads express + mysql2 into node_modules
npm run dev	← starts the server (runs node --watch server.js)
	Lab 10 server on http://localhost:3000
Ctrl + C	← stops the server

Before you start: *Run sql/01, 02 and 03 in Workbench as root. For now, put your root password in db.js; Exercise 3 moves it into a .env file. Open http://localhost:3000 for the front-end.*

Running the Starter Project (Contd.)

The Lab 10 Web App

- Connection: who the app logs in as, and its GRANTs (Ex 2, 3).
- Query plan: EXPLAIN of the search, and the indexes on courses (Ex 1).
- Course search: the server time of every request, plus attack buttons (Ex 1, 3).
- A student's courses: the injection by id (Ex 3).
- Responses: red lines flag an injection that worked or a leaked database error.



Running the Starter Project (Contd.)

If it does not start, read the first error line:

- **Cannot find module 'express'** → you skipped npm install (or ran it in another folder).
- **Access denied for user 'root'** → put your own password in db.js (later: in .env).
- **Access denied for user ''@'localhost'** → .env not found: name it exactly .env, in the project folder.
- **Unknown database 'css326_lab10'** → run sql/01_university_schema.sql first.
- **Cannot find module 'dotenv'** → Exercise 3: run npm install dotenv first.
- **EADDRINUSE :::3000** → the server is already running in another terminal; stop it with Ctrl + C.

Reading a Query Plan: EXPLAIN (Contd.)

Lab Exercise 1: *Index and EXPLAIN*

- Pick a query you run often with a WHERE on a non-key column.
- EXPLAIN ANALYZE it and note the type, rows and actual time.
- CREATE INDEX on that column, then EXPLAIN ANALYZE again.
- Record how type, rows and time changed; that is your evidence the index helped.

Starter code:

```
EXPLAIN ANALYZE SELECT * FROM courses WHERE title = 'Operating Systems';
CREATE INDEX idx_courses_title ON courses (title);
EXPLAIN ANALYZE SELECT * FROM courses WHERE title = 'Operating Systems';
```

The Cost of Indexes, and Tuning Tips

- Indexes are not free: they speed up reads but slow down writes.
- Every INSERT, UPDATE and DELETE must also update each affected index.
- Indexes take storage, and unused ones are pure overhead.
- So index the columns you actually query, and no more, and measure with EXPLAIN.

The Cost of Indexes, and Tuning Tips (Contd.)

More Tuning Tips:

A few habits keep queries fast beyond indexing.

```
-- select only the columns you need (not SELECT *)
SELECT id, title FROM courses WHERE title = ?;

-- a composite index helps a multi-column filter
CREATE INDEX idx_courses_title_seats ON courses (title, seats);

-- don't wrap an indexed column in a function:
--   WHERE LEFT(title, 8) = 'Database'   <-- can't use the index
--   WHERE title LIKE 'Database%'        <-- can
```

Fetch less, filter on indexed columns directly, and let EXPLAIN confirm the plan.

Security Threats: Injection and Over-privilege

A database holds your most valuable data, so it is a target. Three common risks:

- Injection: a crafted input changes what your SQL does.
- Over-privilege: the app can do far more than it needs, so a breach does more damage.
- Exposure: leaked passwords or error details hand attackers a map. We address all three.

Security Threats: Injection and Over-privilege (Contd.)

SQL Injection: the Danger:

Building SQL by gluing in user input lets that input rewrite the query. Never do this.

```
// UNSAFE: the value becomes part of the SQL text
const q = "SELECT * FROM courses WHERE title = '" + input + "'";

// if input is:   ' OR '1'='1
// the query becomes:
//   SELECT * FROM courses WHERE title = '' OR '1'='1'
// ...which returns every row.
```

The input broke out of the quotes and changed the logic. This is SQL injection.

Parameterised Queries as the Injection Defence

The Fix: Send the SQL and the values separately, with ? placeholders. The value can never be code.

```
// SAFE: the driver keeps value and SQL apart
const [rows] = await pool.execute(
  'SELECT * FROM courses WHERE title = ?',
  [input]
);
```

This is the same ? you have used since Lab 4; now you see why it matters. Always parameterise.

Least Privilege: a Dedicated App User

- Never let your app connect as root; root can drop tables, create users, anything.
- If that account leaks, the attacker inherits all of it.
- Instead, make a user that can do only what the app needs on only the app's database.
- This is least privilege: the smallest set of powers that still gets the job done.

Least Privilege: a Dedicated App User (Contd.)

Creating Users and Granting Privileges:

A Dedicated Account; Create it, then grant just the data operations on just your schema.

```
CREATE USER 'university_app'@'localhost'
  IDENTIFIED BY 'a_strong_password';

GRANT SELECT, INSERT, UPDATE, DELETE
  ON css326_lab10.*  TO 'university_app'@'localhost';

-- no DROP, no GRANT, no other schemas
-- REVOKE ... to take a privilege back
```

Then point your app's db config at university_app instead of root.

Least Privilege: a Dedicated App User (Contd.)

Admin vs App Users:

Different Jobs, Different Users; you (the admin) manage the schema; the app only reads and writes data.

```
-- an admin account for people managing the database
CREATE USER 'lab_admin'@'localhost' IDENTIFIED BY 'admin_password';
GRANT ALL PRIVILEGES ON css326_lab10.* TO 'lab_admin'@'localhost';

-- ALL = every privilege on that schema: CREATE, ALTER, DROP, INDEX ...
-- add WITH GRANT OPTION only if they may grant to others
-- root has ALL on *.* (every database): keep it for setup only
```

GRANT ALL is for people managing the database, never for the app's connection.

Least Privilege: a Dedicated App User (Contd.)

Checking and Taking Back Privileges:

REVOKE Undoes GRANT; GRANT uses TO, REVOKE uses FROM. Check the result with SHOW GRANTS.

```
SHOW GRANTS FOR 'university_app'@'localhost';
-- take one privilege back
REVOKE DELETE ON css326_lab10.* FROM 'university_app'@'localhost';
-- take everything back (the user can still log in, but do nothing)
REVOKE ALL PRIVILEGES ON css326_lab10.* FROM 'university_app'@'localhost';
-- remove the user completely
DROP USER 'university_app'@'localhost';
```

A user left with only GRANT USAGE can log in but cannot touch any data.

Least Privilege: a Dedicated App User (Contd.)

Changing a User:

ALTER USER; change a password, or lock an account without deleting it.

```
-- give the app user a new password
ALTER USER 'university_app'@'localhost' IDENTIFIED BY 'new_strong_password';
-- change your own password (whoever is logged in)
ALTER USER USER() IDENTIFIED BY 'my_new_password'; -- (Do not do this, only for understanding)
-- lock / unlock: the account stays, but cannot log in
ALTER USER 'university_app'@'localhost' ACCOUNT LOCK;
ALTER USER 'university_app'@'localhost' ACCOUNT UNLOCK;
-- rename an account
RENAME USER 'university_app'@'localhost' TO 'uni_app'@'localhost';
```

After a password change, update your app's db config too.

Least Privilege: a Dedicated App User (Contd.)

Switching Between Users:

- In Workbench, each user is its own connection: click + next to MySQL Connections.
- Give it a name, set Username to university_app, then click Test Connection.
- Open that connection to work as that user; your root connection stays as it is.
- In the app, the user is whoever db.js (later .env) logs in as; save the file and the server restarts.

Check in SQL:

```
SELECT CURRENT_USER();   -- who am I connected as?
DROP TABLE enrolments;   -- as university_app: ERROR 1142 (DROP denied)
```

Least Privilege: a Dedicated App User (Contd.)

Lab Exercise 2: *A least-privilege app user*

- Create a new MySQL user just for your app.
- GRANT it only SELECT, INSERT, UPDATE and DELETE on your schema.
- Update your app's connection config to use this user, not root.
- Confirm the app still works, and that the user cannot DROP a table.
- REVOKE DELETE, check it with SHOW GRANTS, then GRANT it back.

Starter code:

```
CREATE USER 'university_app'@'localhost' IDENTIFIED BY '...';
GRANT SELECT, INSERT, UPDATE, DELETE ON css326_lab10.*
  TO 'university_app'@'localhost';
```

Protecting Secrets and Errors

- Keep passwords out of your code: load them from an environment variable (a .env file).
- Never commit secrets to Git; add .env to .gitignore.
- Don't send raw database errors or stack traces to the client; log them, return a generic message.
- And keep validating input (Lab 6): defence works in layers, not from one trick.

Protecting Secrets and Errors (Contd.)

Why a .env File?

- Since Lab 6, your DB password has been typed straight into the code (db.js).
- Push that file to GitHub and anyone can read the password.
- Every machine needs different settings, so hard-coded values mean editing code.
- A .env file keeps settings out of your code, and it never leaves your machine.

Protecting Secrets and Errors (Contd.)

Using a .env File:

Install it once with npm install dotenv, put your settings in .env, then read them with process.env.

```
# .env   (project root)
DB_HOST=localhost
DB_USER=university_app
DB_PASS=a_strong_password
DB_NAME=css326_lab10

# .gitignore
.env
node_modules/
```

```
// db.js  (very first line)
require('dotenv').config();
const pool = mysql.createPool({
  host:     process.env.DB_HOST,
  user:     process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  // ...other options as before
});
```

Same code on every machine; each machine keeps its own .env, and Git never sees it.

Protecting Secrets and Errors (Contd.)

Lab Exercise 3: ***Harden the app***

- ❑ Check every query in your app uses ? placeholders; fix any that build SQL by hand.
- ❑ Switch the connection to your least-privilege user from Exercise 2.
- ❑ Move the DB password into a .env file and read it with process.env.
- ❑ Make sure a failed request returns a clean message, not a raw error.

Starter code:

```
require('dotenv').config();
const pool = mysql.createPool({ /* ... */ password: process.env.DB_PASS });
```

Good Habits; Where We're Going Next

- Index what you query (WHERE/JOIN/ORDER BY); confirm with EXPLAIN; don't over-index.
- Parameterise every value; it is correctness and security at once.
- Connect as a least-privilege user, never root; keep secrets in the environment.
- Validate input, hide internal errors, and treat security as layers, not a single fix.

Good Habits; Where We're Going Next (Contd.)

Where We're Going Next:

- Your app is now correct, fast and reasonably hardened: production-shaped.
- Next we explore powerful features that live inside MySQL itself.
- In Lab 11: stored procedures, functions and triggers, and MySQL's NoSQL/JSON side.

Summary

- Indexes turn full scans into fast lookups; EXPLAIN shows the plan (type ALL vs ref, rows).
- Indexes cost write speed and storage, so index the columns you query, and measure.
- Parameterised (?) queries stop SQL injection; build SQL with values, never by string-joining.
- Use a least-privilege app user (not root), keep secrets in .env, and hide internal errors.
- Complete Exercises 1 to 3 in class: index a query, make an app user, and harden the app.

Thank you & Assignment time…